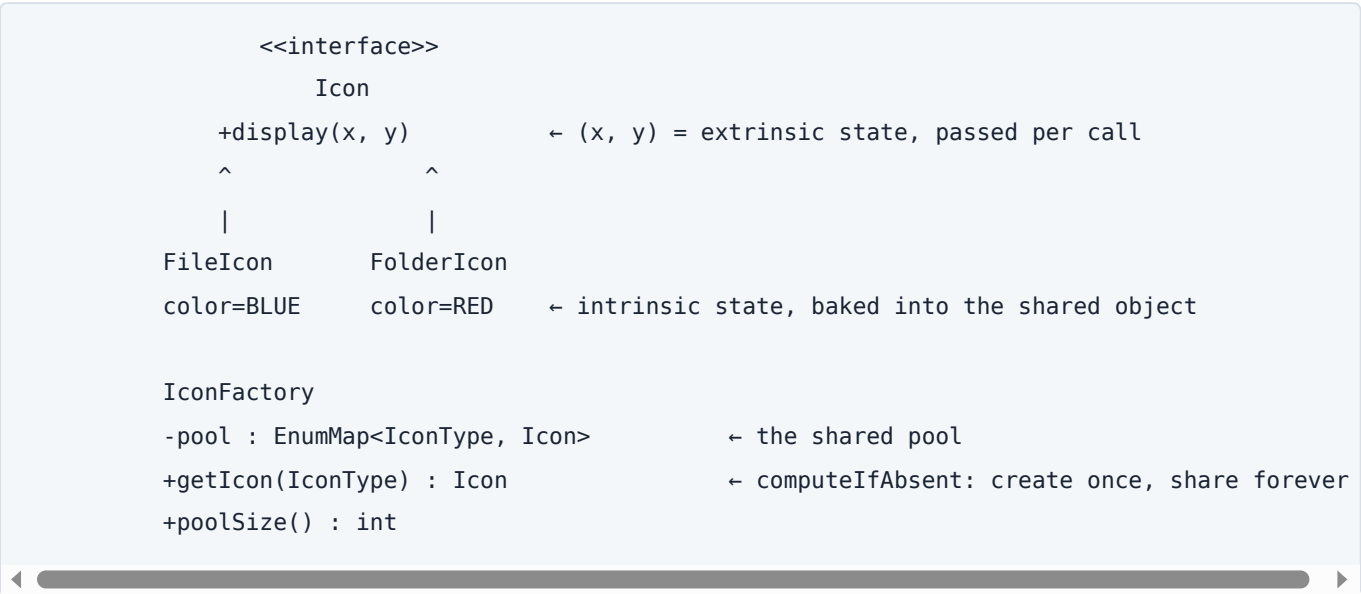


Flyweight Design Pattern

GoF intent: Use sharing to support large numbers of fine-grained objects efficiently. Instead of creating one object per use, the **intrinsic** (shareable, immutable) state lives in a small pool of flyweight objects handed out by a factory, while the **extrinsic** (per-use) state is supplied by the caller at call time.

Structure of this implementation



GoF role	Class in this module
Flyweight (interface)	Icon — display(int x, int y)
ConcreteFlyweight (shared instances)	FileIcon (BLUE), FolderIcon (RED)
FlyweightFactory (creates + pools)	IconFactory — lazy computeIfAbsent over an EnumMap<IconType, Icon>
Client	FlyweightDesignPattern.main()

How it works

1. The client only ever asks the factory: `iconFactory.getIcon(IconType.FILE)`. It never calls `new FileIcon()` itself.
2. On the **first** request for a type, the factory creates the flyweight and caches it (`computeIfAbsent`, logged as `(pool miss)`). Every later request returns the **same shared instance**.
3. Intrinsic state (the icon's color/glyph) lives inside the flyweight and is immutable; extrinsic state (where to draw it) is passed to `display(x, y)` by the caller on every call.

4. Result: any number of placements cost only one object per icon *type* — 1,000 files on screen = 1 `FileIcon`.

Verified output (`java com.design.patterns.flyweight.FlyweightDesignPattern`):

```
Flyweight Design Pattern
(pool miss) creating flyweight for FILE
----Drawing BLUE FileIcon at (10,20)
----Drawing BLUE FileIcon at (30,40)
(pool miss) creating flyweight for FOLDER
----Drawing RED FolderIcon at (50,60)
----Drawing RED FolderIcon at (70,80)
FILE flyweight reused (file1 == file2): true
Objects in pool for 4 placements: 2
```

Note the two proofs in the output: only two `(pool miss)` lines despite four `getIcon` calls, and `file1 == file2` is `true` — the same object, not an equal copy.

Why this follows the pattern

- **✓ Sharing is real and demonstrated:** one instance per type; repeat requests return aliases (`==` verified), never copies.
- **✓ Intrinsic vs. extrinsic split:** color lives in the shared flyweight; coordinates are supplied per call through `display(x, y)`.
- **✓ Factory owns creation:** flyweights are built lazily on first request; clients cannot bypass the pool.
- **✓ Same shape as real-world flyweights:** `Integer.valueOf()` cache, `String` interning, glyph/icon caches in UI toolkits.

History

The first version pre-built the instances in the client, keyed them by *color*, and had no extrinsic state (`display()` took no arguments) — making it a lookup registry rather than a flyweight. Fixed on 2026-07-08: lazy factory creation, `IconType` keys (`FILE` / `FOLDER`), per-call `(x, y)` extrinsic state, and removal of unused Spring/Jackson annotations and the `IconEmums` typo.